School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00171248
Student Name:	Lexie Walsh

Module Information

Module Title:	Security Architecture
Module Code:	COMPH2701

Assessment Information

Assessment Title:	Designing and Implementing a Proof of Concept Cloud Solution based on a clients needs

Declaration

✓ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

✓ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

✓ I declare that this assessment submission is my own work.

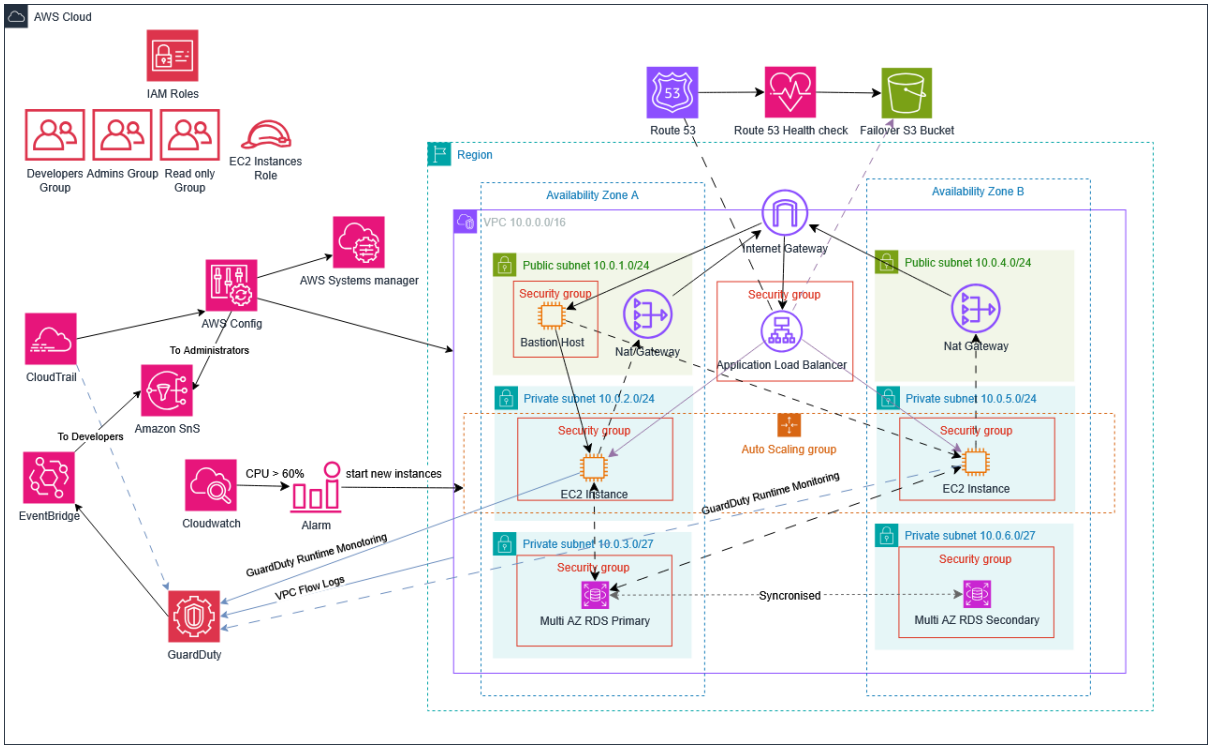
✓ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Lexie Walsh
Date:	01.05.2026

Table of Contents

Task 1: Create a detailed AWS Architecture Diagram 1
Task 2: Proof of Concept 2
Network design: 2
Bastion Host 3
Amazon Machine Image (AMI) 4
Single-AZ Amazon RDS Deployment 4
Auto Scaling Configuration 6
Load balancer configuration 7
Security Implementation 9
Static Backup Website 11
Task 3: Justification of Additional AWS Services 12
Service 1 – AWS Config 12
Service 2 – GuardDuty 12
Task 4: Sustainability 13
References: 15

Task 1: Create a detailed AWS Architecture Diagram

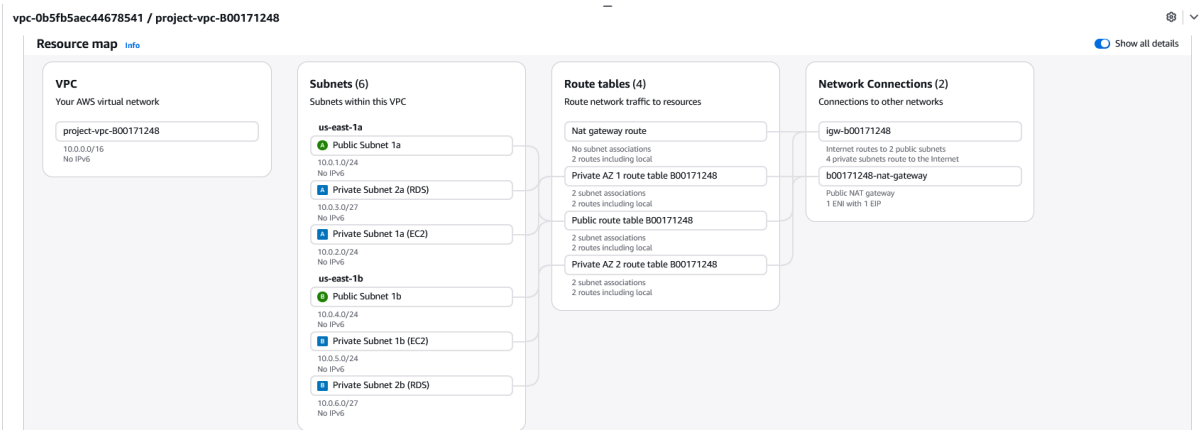


Task 2: Proof of Concept

In the proof of concept, a simplified version of the Task 1 Diagram was designed to demonstrate how the network environment would look if this design would be implemented:

Network design:

For the Network design, a VPC was created in the 10.0.0.0/16 range. 2 public and 4 private subnets were created ranging from 10.0.1.0/24 to 10.0.6.0/27 across two availability zones. The public subnets are used for bastion host(s), whereas the private subnets are used for the EC2 instances used for the website and database connectivity using amazon RDS.

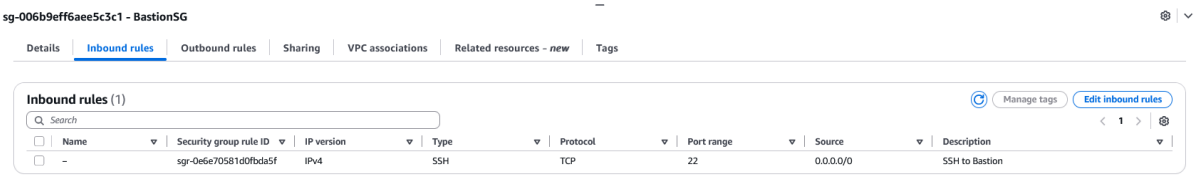


The public subnets have a public route table going directly to the internet gateway. Each private subnet has its own route table to the NAT gateway.

Bastion Host

A bastion host was created in a public subnet with a public ipv4 address, allowing ssh access to the web EC2 instances, which all are in the private subnets 1a and 1b.

A security group was created specifically for the bastion allowing any ssh traffic to enter the Bastion instance. This way, only ssh traffic is allowed inbound.



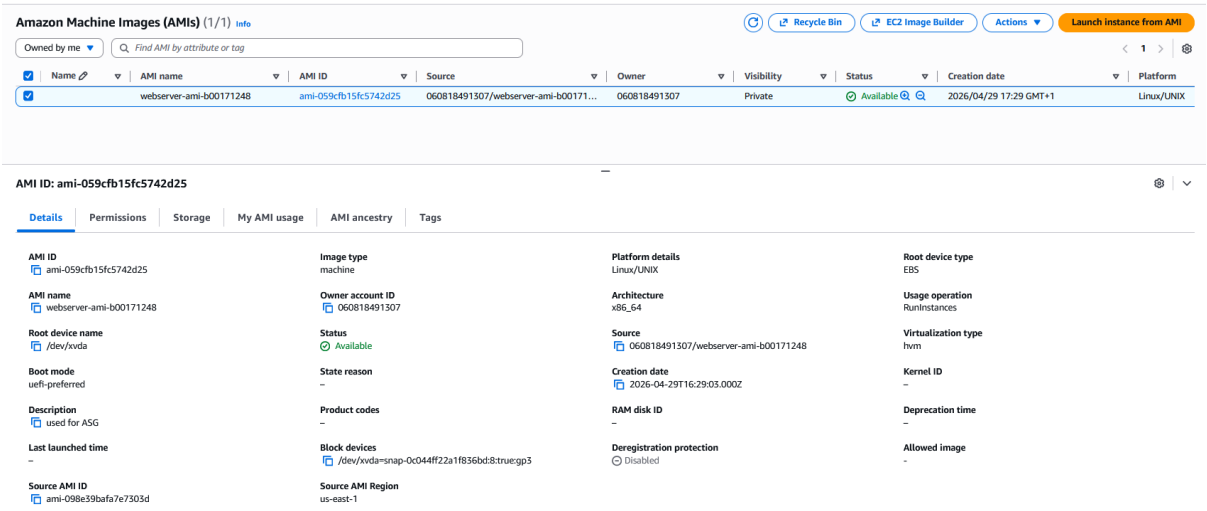
The EC2 security group's inbound traffic only allows inbound ssh traffic from the bastion host security group. This makes it that the only way it is possible to access the EC2 web-hosts is through a separate EC2 bastion host:

```
C:\Users\alexc>ssh -A -i "D:\TU Dublin Work\Security Architechture\.Project\labsuser.pem" ec2-user@44.211.209.16
The authenticity of host '44.211.209.16 (44.211.209.16)' can't be established.
ED25519 key fingerprint is SHA256:LuGa8Ud/u7wEzyLMdP1QesdqNt4fWHsUZ1asAz6D7Gc.
This host key is known by the following other names/addresses:
    C:\Users\alexc/.ssh/known_hosts:9: 3.89.140.147
    C:\Users\alexc/.ssh/known_hosts:11: 54.210.9.184
    C:\Users\alexc/.ssh/known_hosts:12: 18.204.222.94
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.211.209.16' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Apr 29 16:03:51 2026 from 37.228.213.141
[ec2-user@ip-10-0-1-151 ~]$ ssh ec2-user@10.0.2.63
The authenticity of host '10.0.2.63 (10.0.2.63)' can't be established.
ED25519 key fingerprint is SHA256:Be6xuwA0n+308CCXJKS/je3cbeOmDa8+pS5fGNp9QXA.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.63' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Apr 29 16:04:05 2026 from 10.0.1.151
[ec2-user@ip-10-0-2-63 ~]$
```

A bastion host is used to give admins and developers a single secure point of access, without having to SSH into the webserver directly, which would allow attackers direct access to the webserver when breached. This entry point can be used to log and monitor who accessed the bastion host, what they did (eg. SSH) and when they did it.

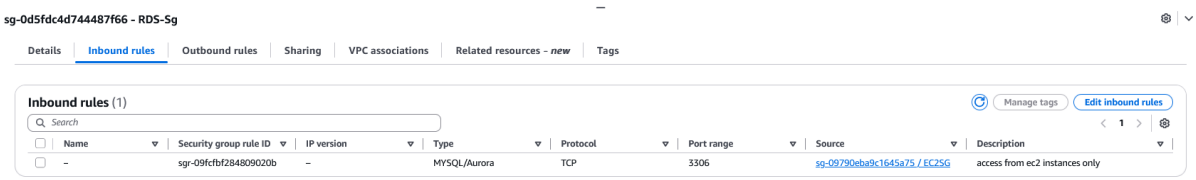
Amazon Machine Image (AMI)

An amazon machine image is a copy of a configured instance. For this, the test-instance was configured and copied, to be used in autoscaling to ensure that every instance in autoscaling has all configurations the company may need for the website to run securely.



Single-AZ Amazon RDS Deployment

For the proof of concept, a Single-AZ RDS was deployed to show an example of how user data can be stored in a relational database. For better scalability, a multi AZ (availability zone) model can be implemented, making one RDS instance the primary one and the other one a secondary one, used in case an availability zone goes down or there is too much traffic going to the primary rds. They are both automatically synchronized with each other. The RDS has a security group allowing it to only accept access from EC2 instances within the EC2 security group:



This allows exclusive access from the web EC2 instances to the RDS.

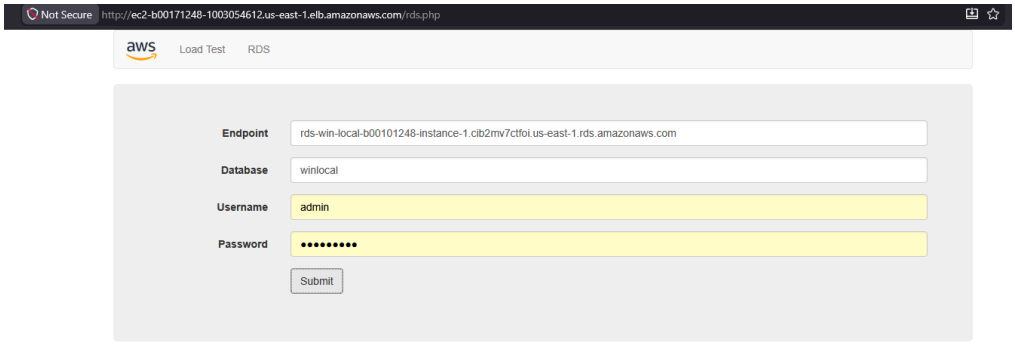
```
C:\Users\alexc>ssh -A -i "D:\TU Dublin Work\Security Architechture\.Project\labsuser.pem" ec2-user@44.
211.209.16
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Apr 29 18:01:24 2026 from 37.228.213.141
[ec2-user@ip-10-0-1-151 ~]$ ssh ec2-user@10.0.2.63
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Apr 29 18:03:19 2026 from 10.0.1.151
[ec2-user@ip-10-0-2-63 ~]$ mysql -h rds-win-local-b00101248-instance-1.cib2mv7ctfoi.us-east-1.rds.amaz
onaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 2382
Server version: 5.7.12 MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

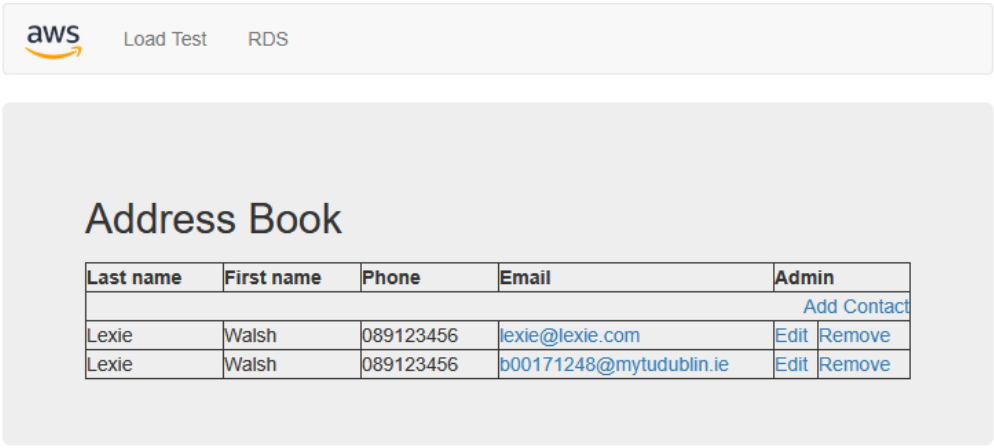
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

The test website demonstrates that it is possible to login to the RDS through a web page too:

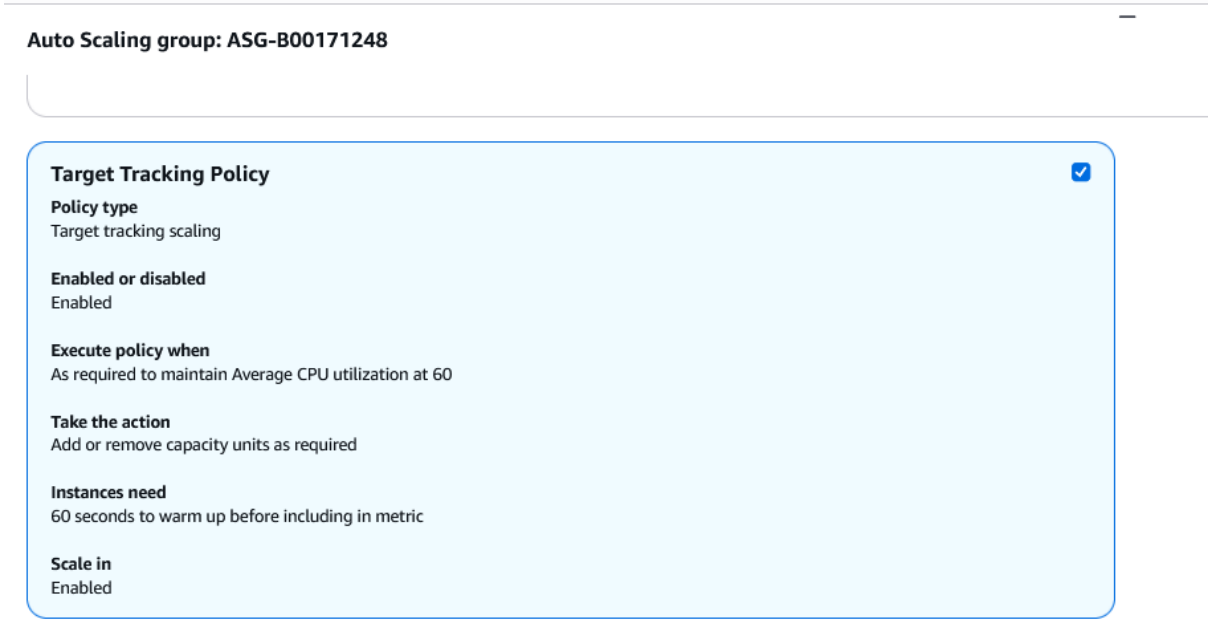


When entering the endpoint, the test created database (called winlocal), and the RDS username and password, it enables a view of the Address Book table:

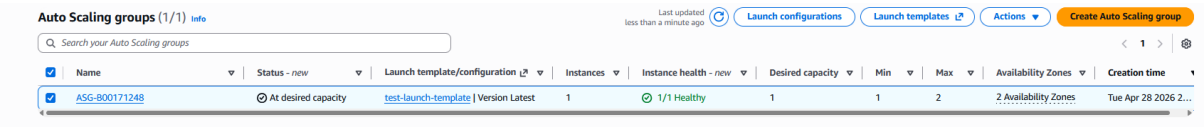


Auto Scaling Configuration

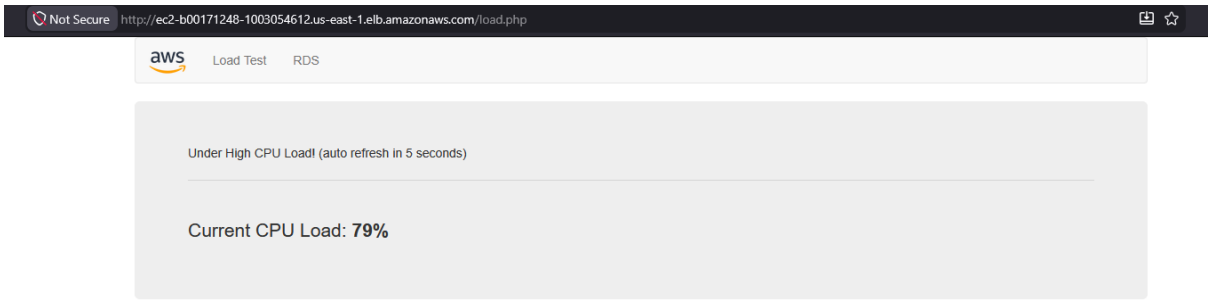
Autoscaling was configured using the AMI created of the test web instance with a desired capacity of 1, minimum capacity of 1 and maximum capacity of 2. This can be adjusted as needed for the average demand of winlocal. To confirm with the requested CPU utilization, a Target Tracking policy was created to fulfil the request to maintain an average CPU utilisation across all instances at a maximum of 60%.



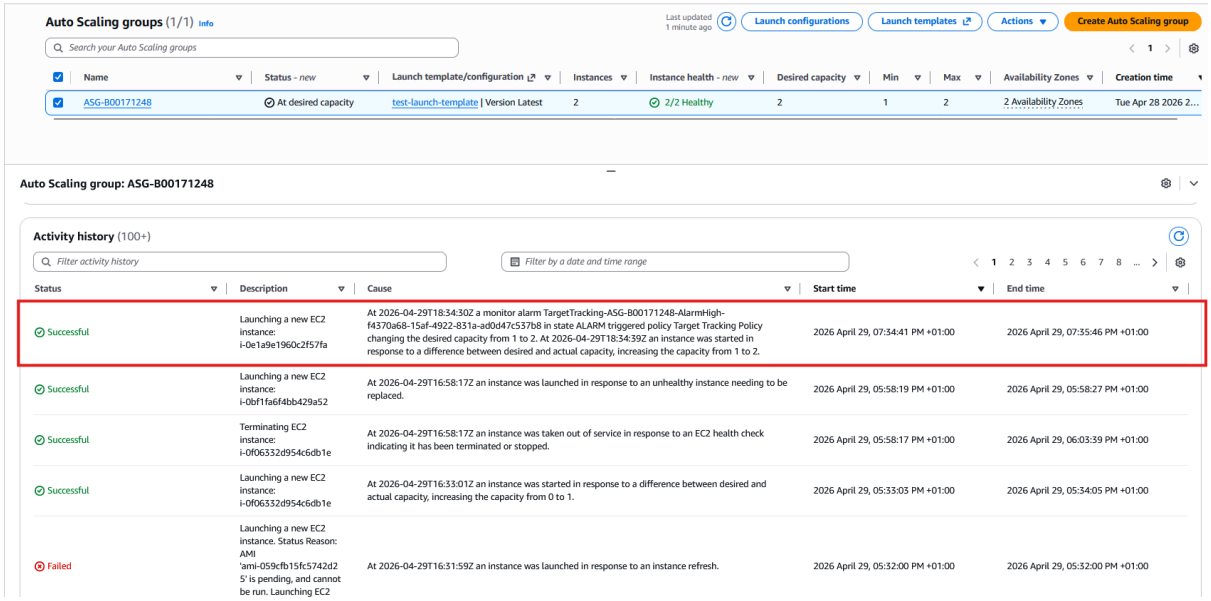
When initially starting the Autoscaling group, it starts 1 instance, reaching the desired capacity.



The test AWS web page that was added, has a feature to put a load on the ec2 instances using the Load Test feature. This will increase the CPU load drastically, making it possible to test the Auto scaling policy:



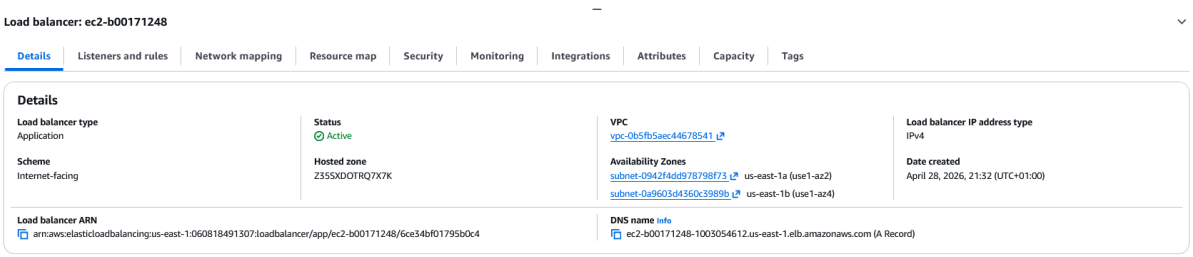
A monitor Alarm is triggered and changed the capacity from 1 instance to 2 instances. If the load decreases again, this will be reduced back to 1 instance.



Load balancer configuration

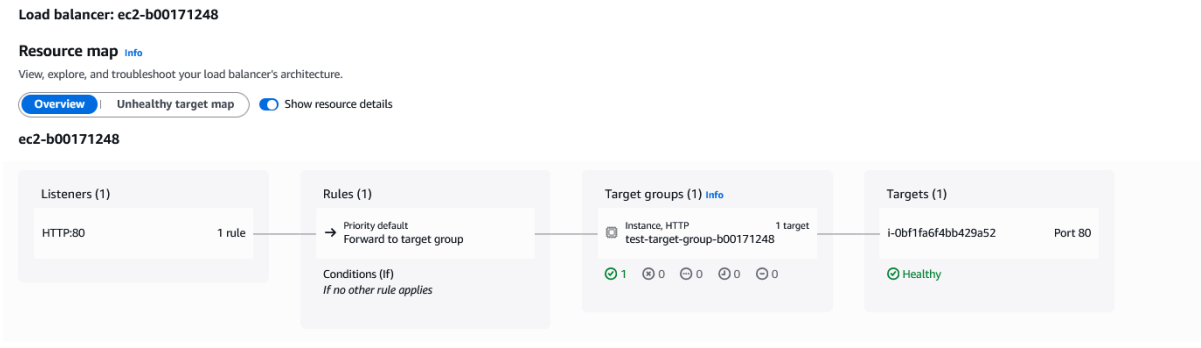
An application load balancer (ALB) was selected and configured to load traffic between multiple availability zones. The ALB was selected because WinLocal giveaways works entirely on HTTP/S requests and web hosting, rather than other networking protocols such as UDP and TLS.

To configure the ALB, the subnets created for the ec2 web instances in each AZ are used. For the proof of concept, this was only implemented using HTTP, rather than HTTPS. This is because using HTTPS connections require a signed certificate. This can easily be implemented to support HTTPS too.

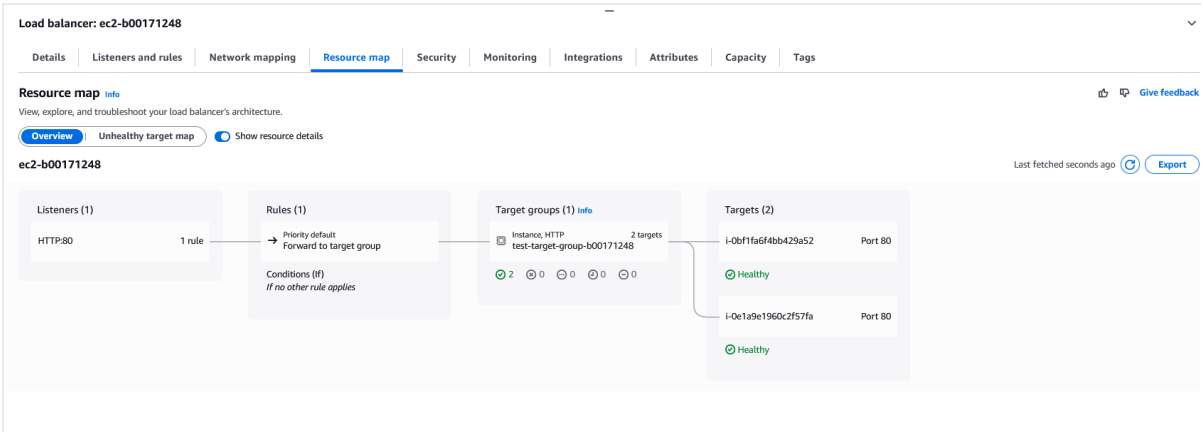


The load balancer creates a DNS name, which is used to publicly access the ec2 instances. This is how users will access the web page. The DNS entry page is used in the proof of concept to put a load on the ec2 instance and to access the relational database (RDS) for testing.

Before loading the webserver using the load feature, the load balancer only had one instance in it, which was initiated into the ec2 subnet in the AZ, us-east-1a.



After the auto scaling group initiates a second instance, the load balancer loads the instance into the subnet in the second AZ, us-east-1b.

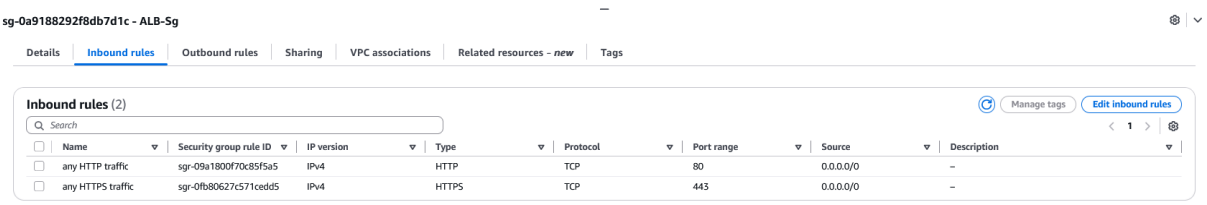


Having the load balancer scale in multiple availability zones increases availability. In the case one AZ has technical difficulties or goes down, the website won't go down, as all traffic will jump to the other available zone.

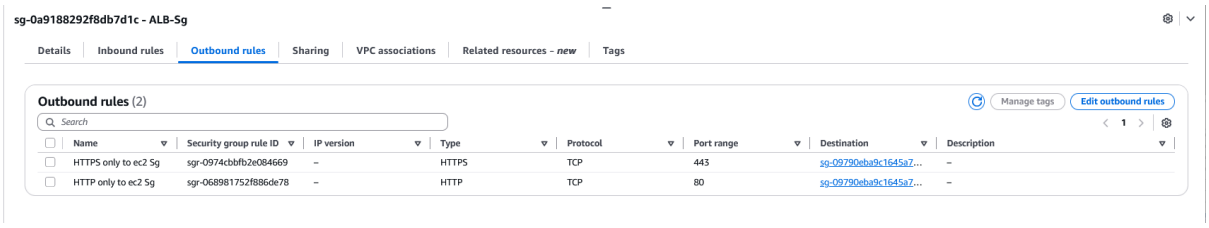
Security Implementation

For the proof of concept, subnets and security groups were created to provide network segmentation:

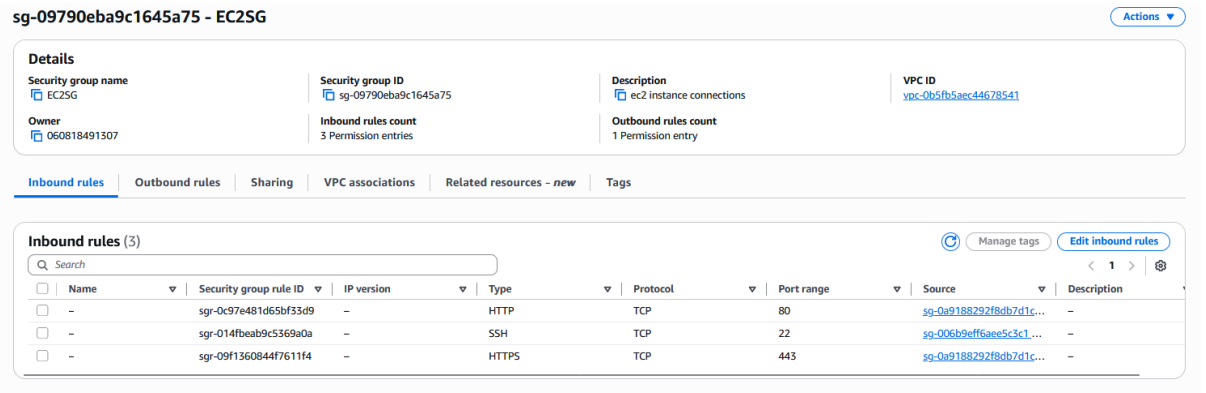
The Access load balancer security group (ALB-Sg) specifically allows any HTTP/HTTPS traffic to enter from any source.



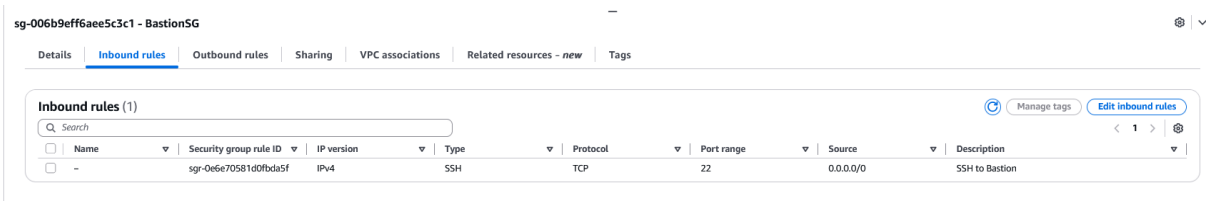
The outbound rules only allow traffic to go to the EC2 security group (EC2SG). This allows user traffic to enter through the load balancer and be transferred to one of the EC2 instances.



The EC2 security group (EC2SG) allows HTTP/S requests from the application load balancer security group (ALB-Sg) and SSH access from the bastion's security group (BastionSG) This is to only allow direct access to the EC2 cli through the bastion host.



The bastion host security group (BastrionSG) only allows SSH inbound requests, to allow developers or other authorised staff to gain access to the EC2 instances.



Currently it is set to all ipv4 addresses. This can be changed to a network range that can for example only be accessed when using a VPN connection. This reduces the risk of attackers accessing the bastion host.

Lastly, a security group for RDS was created (RDS-Sg). This security group ensures that only the EC2 instances are allowed to communicate with the Database.



To ensure proper segmentation, the bastion host is the only instance in the public subnets. The web servers and RDS databases are each in their own separate subnet groups. Each availability zone has their own set of 3 subnets (1 public 2 private).

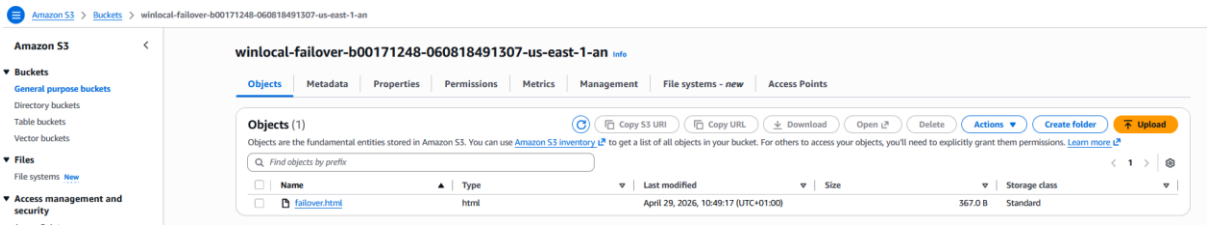


Additionally, the RDS subnets are reduced to /27, rather than /24. This was done because an RDS doesn't need 251 (5 reserved IP addresses) to run an RDS. Depending on the RDS class size WinLocal decides on, it's unlikely to use more than 25 IP addresses between its writer and reader databases. If WinLocal ever gets popular enough to need more addresses, the subnet range and RDS class size can be adjusted as needed.

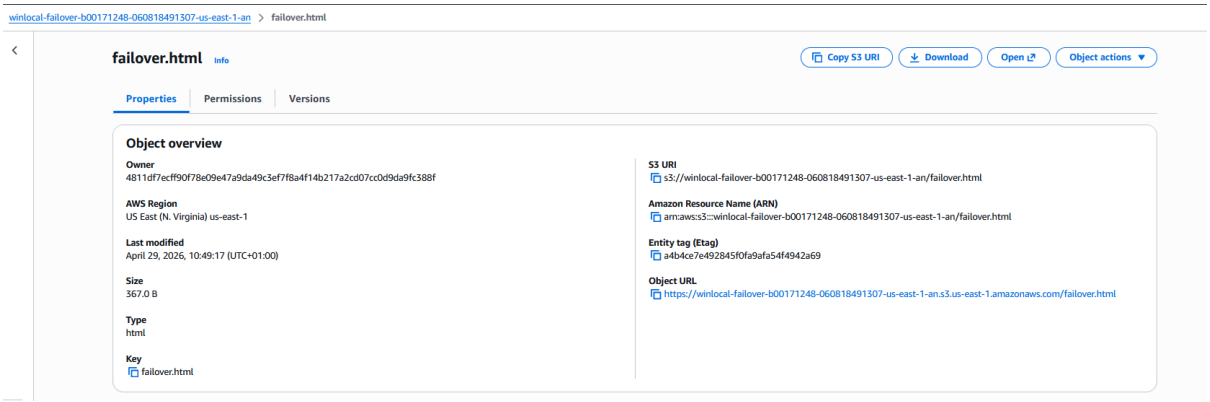
In the Proof of Concept, only security groups and subnetting was configured to provide network segmentation. This is sufficient for a PoC, but in real world systems it is recommended to additionally set up ACL's (Access Control Lists). These Lists allow the developers to specifically allow and deny traffic, allowing more control to which services are allowed and explicitly denied. The ACL rule set can additionally be ordered in which manner traffic is allowed/denied, allowing for example no ssh traffic through, but from the specific IP address range a VPN service offers.

Static Backup Website

A static backup website was created and is hosted on an Amazon S3 bucket called Failover. The backup website page is used in the case that the website goes down for maintenance or any other reasons and gives the customer a message explaining that the website is experiencing technical difficulties and ask them to try again later. This is done by an AWS service called Rule53, which in case the website goes down, redirects web requests to the failover page. The AWS Academy Learner Lab does not allow a configuration of Rule53. Therefore, only a S3 bucket with a simple HTML web page was created to demonstrate how this feature would work.



Failover.html was uploaded to the winlocal-failover bucket and when opening it, an object URL is given: https://winlocal-failover-b00171248-060818491307-us-east-1-an.s3.us-east-1.amazonaws.com/failover.html



When opening the object URL, a simple web page appears:



Task 3: Justification of Additional AWS Services

Service 1 – AWS Config

AWS Config is a service that when set up will monitor and manage resource configuration changes. This tool allows the developers define rules stating what configuration is compliant for each resource. For example, the RDS security group only allows communication from EC2 webservers. If an AWS Config rule is created backing this up and a developer/hacker were to attempt to edit this rule to allow communication from any source instead of only the EC2 Security group, AWS Config would flag this as non-compliant. This non-compliant rule can be sent to AWS Systems Manager, which can revert the change back to the config rule, reversing the change.

CloudTrail allows AWS Config to send alarms and log any attempted changes to any service, giving full transparency to when and who attempted to change a setting that is monitored by AWS config.

This service can additionally be directly linked to Amazon SnS (Simple Notification service), allowing any changes flagged as unauthorised to be sent to administrators via email. This directly addresses the poor risk management identified in the 10 week breach, allowing for a quick identifier that the system might be breached.

Service 2 – GuardDuty

AWS GuardDuty is a service which uses AI and machine learning to detect AWS accounts, workloads and data threats. It constantly monitors AWS accounts and workloads for any malicious activity it monitors CloudTrail management events, VPC Flow logs and DNS query log analysis. Additionally, it can scan EC2 instances for malware/unusual activity. GuardDuty generates alerts for any findings, which can trigger AWS EventBridge to send an email to admins using AWS SNS. This will work alongside AWS Config and CloudWatch, giving extra protection.

This service would have increased the chance of detecting the ransomware attackers lateral movement across the network and the data exfiltration during the 10 week breach.

GuardDuty continuously monitors systems 24/7 without needing human intervention. This reduces the number of logs humans need to review. WinLocal doesn't have the manpower required to manually monitor logs continuously. Guard duty can fill this gap by reducing the amount of reviewing required, saving WinLocal operation costs without compromising security.

Task 4: Sustainability

The AWS Well-Architected framework sustainability pillar is a good way to guide WinLocal onto a sustainable path. The proof of concept was optimised using the sustainability pillars guidance.

Auto scaling was implemented to reduce unused resources. This service will scale instances to fit the number of resources used by the users accessing the resource, reducing the energy consumption behind the running resources.

AWS managed services such as RDS and ALB (Application Load Balancer) were implemented. AWS manages and optimises the infrastructure of those, which is more efficient than WinLocal running their own database server on premises, where the server was running regardless of demand.

The Proof of Concept itself is part of the sustainability pillar of the AWS Well-Architected framework. It is recommended to create the underlying structure on a small scale, allowing for a safe way to test how many resources are needed. This information is a great way to test the system's infrastructure before deploying it with maximum resources. This allows WinLocal to create a baseline of what resource requirements (instance type, storage requirements, etc.) are needed for WinLocal to run efficiently, whilst reducing the amount of unused resources.

For future improvements, WinLocal could consider moving old competition data from the RDS to cheaper storage options using services like Amazon S3 Lifecycle. Some customer information may need to be stored for compliance reasons, but don't need to get accessed by personnel. The S3 Lifecycle will keep track of how long data is stored and after an initially set up time period, will automatically delete or archive this information.

Another improvement WinLocal could add, is the AWS Sustainability Service. This tool allows companies to track, measure, review and forecast carbon emissions of the generated AWS usage and is verified by AWS, which follows the greenhouse gas (GHG) protocol and its underlying standard ISO 14064. This feature is useful to promote how sustainable WinLocal is, track sustainability goals of the company and how emissions are considered in operational workflows.

As a recommendation for the future, WinLocal could consider migrating into a serverless environment to reduce the carbon footprint even more. WinLocal's primarily focus lies on collecting customer data to enter competitions and promoting their sponsors. With the high variability of traffic to the website depending on if a competition is being hosted, a serverless approach might be more appropriate in the long run. This a big change to the PoC, which uses EC2 instances and an ALB and would require a significant redesign of the architecture currently in place. Serverless would run using

AWS Lambda functions and API gateways to send http requests to Lambda functions. This approach uses resources only when a user actively attempts to access a web page, rather than running an EC2 instance 24/7.

References:

docs.aws.amazon.com. (n.d.). *Configuring DNS failover - Amazon Route 53*. [online] Available at: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover-configuring.html.

Amazon.com. (2025). *How health checks work in simple Amazon Route 53 configurations - Amazon Route 53*. [online] Available at: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover-simple-configs.html.

Amazon.com. (2024). *Amazon Route 53 | AWS Architecture Blog*. [online] Available at: https://aws.amazon.com/blogs/architecture/category/networking-content-delivery/amazon-route-53/.

docs.aws.amazon.com. (n.d.). *Bucket policy examples - Amazon Simple Storage Service*. [online] Available at: https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html?icmpid=docs_amazons3_console.

aws.amazon.com. (2022). *New for Amazon GuardDuty – Malware Protection for Amazon EBS Volumes | AWS News Blog*. [online] Available at: https://aws.amazon.com/blogs/aws/new-for-amazon-guardduty-malware-detection-for-amazon-ebs-volumes/.

Amazon.com. (2026). *GuardDuty Runtime Monitoring - Amazon GuardDuty*. [online] Available at: https://docs.aws.amazon.com/guardduty/latest/ug/runtime-monitoring.html.

docs.aws.amazon.com. (n.d.). *How Runtime Monitoring works with Amazon EC2 instances - Amazon GuardDuty*. [online] Available at: https://docs.aws.amazon.com/guardduty/latest/ug/how-runtime-monitoring-works-ec2.html.

Amazon.com. (2024). *Sustainability - AWS Well-Architected Framework*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/framework/sustainability.html.

docs.aws.amazon.com. (n.d.). *Sustainability Pillar - AWS Well-Architected Framework - Sustainability Pillar*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html.

Amazon.com. (2023). *SUS05-BP01 Use the minimum amount of hardware to meet your needs - Sustainability Pillar*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sus_sus_hardware_a2.html [Accessed 1 May 2026].

Amazon.com. (2021). *SUS05-BP03 Use managed services - Sustainability Pillar*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sus_sus_hardware_a4.html [Accessed 1 May 2026].

Amazon.com. (2026). *Test and validate improvements - Sustainability Pillar*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/test-and-validate-improvements.html [Accessed 1 May 2026].

Amazon.com. (2023). *SUS04-BP02 Use technologies that support data access and storage patterns - Sustainability Pillar*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sus_sus_data_a3.html [Accessed 1 May 2026].

docs.aws.amazon.com. (n.d.). *Managing your storage lifecycle - Amazon Simple Storage Service*. [online] Available at: https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html.

Amazon.com. (2023). *SUS06-BP01 Communicate and cascade your sustainability goals - Sustainability Pillar*. [online] Available at: https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sus_sus_dev_a1.html [Accessed 1 May 2026].

Amazon.com. (2026). *What is AWS Sustainability? - AWS Sustainability*. [online] Available at: https://docs.aws.amazon.com/sustainability/latest/userguide/what-is-sustainability.html#calculation-methodology-overview [Accessed 1 May 2026].

Amazon.com. (2026). *Calculation methodology - AWS Sustainability*. [online] Available at: https://docs.aws.amazon.com/sustainability/latest/userguide/methodology.html [Accessed 1 May 2026].